



HISTORIA DEL LENGUAJE DE PROGRAMACION DE

Python

Programación II · Investigación #1

Integrante: Adrian Jiménez 4-839-2413 **Profesor:** Napoleon Ibarra Primer Semestre 2026

El Origen de Python



Nacimiento

Concebido por Guido van Rossum a finales de los años 80 en los Países Bajos (CWI). Implementación comenzó en 1989.



Primera versión

La versión 0.9.0 se publicó en febrero de 1991, creada como proyecto personal de Guido durante sus vacaciones de Navidad.



Objetivo inicial

Crear un sucesor del lenguaje ABC que manejara excepciones e interactuara con el sistema operativo Ameba.



¿Por qué "Python"? No proviene del reptil — el nombre fue tomado del grupo cómico británico Monty Python, del que Guido era fanático. Buscaba un nombre corto y misterioso.

Evolución y Versiones Clave



Python 1.0

1994

Introdujo funciones lambda, map, filter y reduce.



Python 2.0

2000

Sumó listas por comprensión y recolección de basura. Base de proyectos heredados.



Python 3.0

2008

Corrección de fallos fundamentales. No compatible con v2. Estándar absoluto hoy (fin de soporte v2: 2020).



Python 3.13+

2024-2025

Más rápido y estable que nunca. Mejoras en el GIL y rendimiento general.

Concepto y Características

Python es un lenguaje de programación de propósito general, de alto nivel y con semántica dinámica. Prioriza la legibilidad del código, permitiendo expresar conceptos complejos en pocas líneas.



Sintaxis legible

Similar al inglés;
facilita aprendizaje y
lectura.



Interpretado y Dinámico

No necesita
compilación; tipos
asignados en
ejecución.



Multiparadigma

OOP, estructurado o
funcional según la
necesidad.



Open Source

Mantenido por
Python Software
Foundation. Gratuito.



"Baterías Incluidas"

Biblioteca estándar
enorme para casi
cualquier tarea.

Importancia y Relevancia



Democratización

Rompió la barrera de entrada para no-ingenieros. Científicos, biólogos y economistas automatizan tareas sin ser expertos en computación.



Motor de la IA

Lenguaje más importante en IA y Machine Learning. TensorFlow y PyTorch están diseñados para Python — base de ChatGPT y conducción autónoma.



Eficiencia (Time-to-Market)

Empresas como Google, NASA y Netflix usan Python por la velocidad en prototipado. Reduce costos y tiempo de desarrollo significativamente.

Listas vs. Tuplas

	Listas []	Tuplas ()
Mutabilidad	Mutable: puede cambiar después de su creación.	Inmutable: no se puede modificar una vez creada.
Longitud	Dinámica: crece o decrece según sea necesario.	Fija: constante durante toda la ejecución.
Métodos	append(), extend(), insert(), pop(), remove()...	Solo consulta: count(), index().
Uso típico	Datos homogéneos (misma naturaleza).	Datos heterogéneos o registros fijos.
Eficiencia	Más memoria por su naturaleza dinámica.	Más ligera y rápida en memoria.

Diccionarios vs. Conjuntos

Diccionario { clave: valor }

Implementación de un mapa/arreglo asociativo. Usa función hash para convertir clave inmutable en dirección de memoria donde se guarda el valor.

Usos:

Manejo de datos de APIs web (JSON)
Contadores o tablas de frecuencia
Estructura base del lenguaje Python

Estructura:	Tabla Hash (pares clave-valor)
Acceso:	Mediante clave única
Duplicados:	Claves únicas; valores repetibles
Orden:	Mantiene orden de inserción (Python 3.7+)

Conjunto { elemento }

Colección mutable de elementos inmutables y únicos. Basada en teoría matemática de conjuntos. Operación de pertenencia $O(1)$ — casi instantánea.

Usos:

Eliminación eficiente de duplicados
Operaciones algebraicas (unión, intersección)
Verificación de membresía ultrarrápida

Estructura:	Tabla Hash (solo claves/valores)
Acceso:	Sin índice ni clave directa
Duplicados:	No permite duplicados
Orden:	Desordenado

Métodos para Manipulación de Listas

+ Adición

`append(x)`

Añade x al final de la lista.

`insert(i, x)`

Inserta x en la posición i.

`extend(lista2)`

Fusiona otra lista al final.

- Eliminación

`pop(i)`

Elimina y retorna el elemento en posición i.

`remove(x)`

Elimina la primera aparición del valor x.

`clear()`

Vacía la lista por completo.

Organización y Análisis

`sort()`

Ordena elementos de forma ascendente.

`reverse()`

Invierte el orden actual.

`index(x)`

Retorna la posición del valor x.

`count(x)`

Cuenta cuántas veces aparece x.

`copy()`

Crea copia independiente de la lista.

Conclusiones

01

Python nació de la necesidad de un lenguaje legible y accesible. Su origen humilde como proyecto personal se convirtió en el fundamento de la programación moderna.

02

Su filosofía de "código abierto" y "baterías incluidas" lo hizo accesible a todo tipo de profesionales, democratizando la programación más allá de la ingeniería.

03

Las estructuras de datos nativas (listas, tuplas, diccionarios, conjuntos) ofrecen herramientas precisas para cada problema, equilibrando flexibilidad y eficiencia.

04

Hoy Python lidera el campo de la Inteligencia Artificial y el Machine Learning, siendo la base de tecnologías como TensorFlow, PyTorch y ChatGPT.